

El proyecto *Musa Jeans* consiste en una arquitectura basada en microservicios desarrollada con Spring Boot, orientada a la gestión de una tienda de jeans. El sistema está dividido en servicios independientes que permiten gestionar clientes, productos (jeans), pagos y envíos.

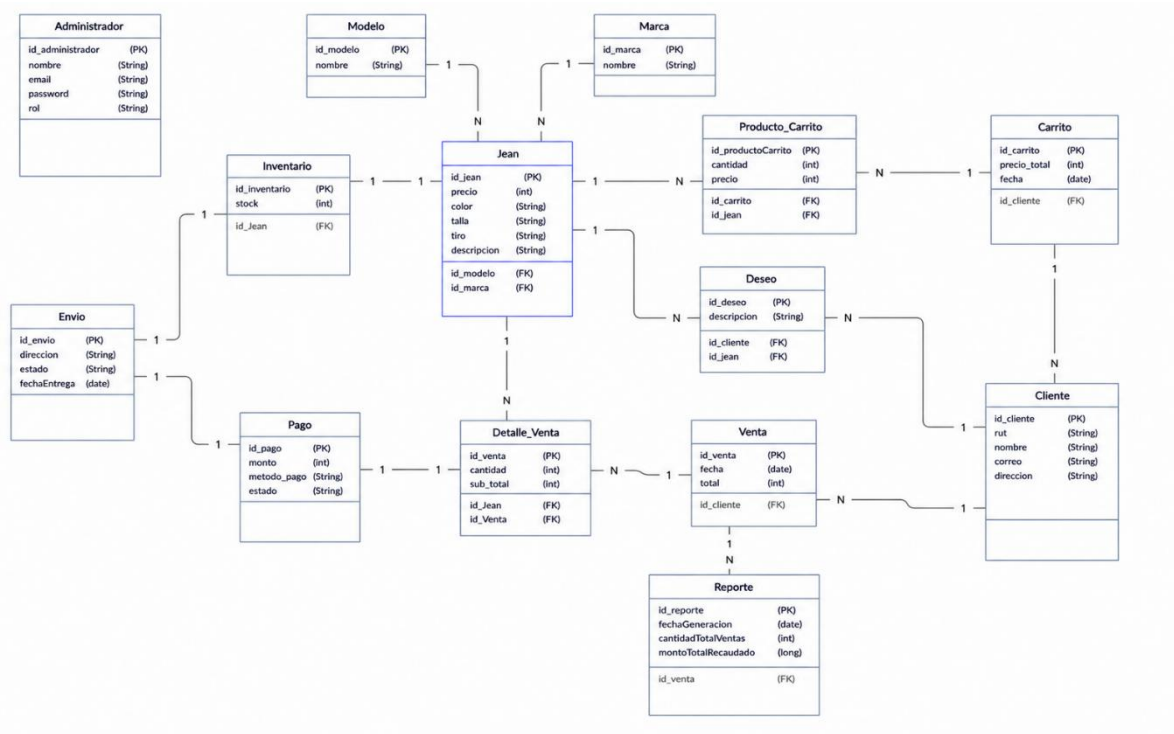
Cada microservicio es independiente y cuenta con su propia capa de:

- Controller
- Service
- Repository
- Model

Esto permite escalabilidad, mantenimiento independiente y separación de responsabilidades.

El sistema utiliza una base de datos relacional MySQL y comunicación entre servicios mediante WebClient.

Diagrama modelo base de datos



Servicio Autenticación

- **¿En qué consiste?**

El servicio de autenticación es responsable de validar la identidad del usuario y controlar el acceso al sistema. Permite asegurar que solo usuarios autorizados puedan consumir los endpoints de los microservicios.

- **¿Cómo se implementa la autenticación por token?**

Se implementa mediante **JWT (JSON Web Token)**:

1. El usuario envía credenciales (usuario/contraseña)
2. El servidor valida los datos
3. Si son correctos, genera un token JWT
4. El cliente usa ese token en cada petición

Implementación por capas

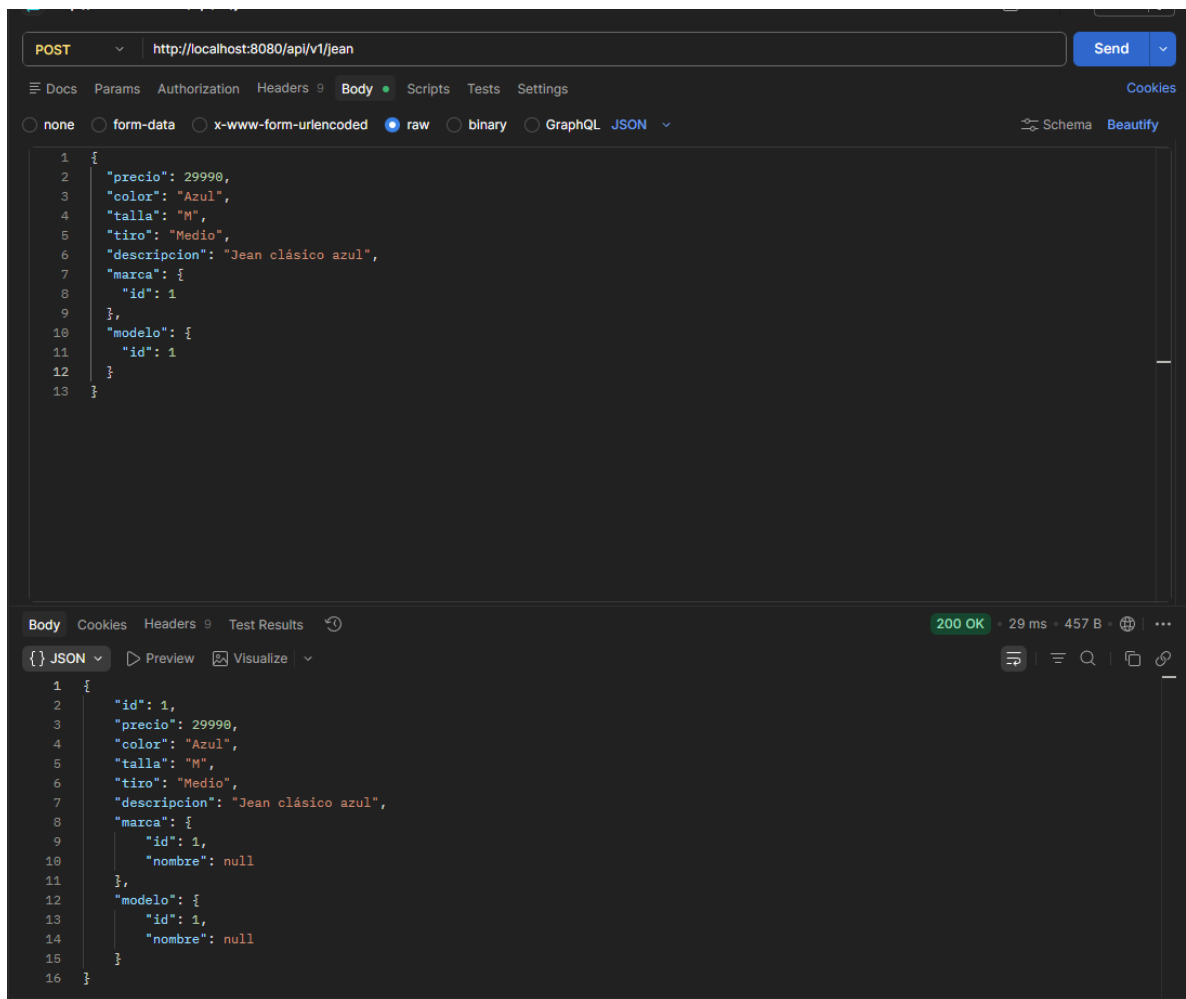
- **¿Cómo funciona la arquitectura por capas?**

La arquitectura por capas es una forma de organizar el sistema en distintas partes, donde cada una cumple una función específica para mantener el código ordenado y fácil de mantener. Spring Boot normalmente se divide en cuatro capas:

- **Controller** es la encargada de recibir las peticiones HTTP desde el cliente
- **Service** contiene la lógica de negocio del sistema
- **Repository** es la encargada de comunicarse directamente con la base de datos
- **Model** representa las entidades del sistema, es decir, las clases que se relacionan con las tablas de la base de datos.

En conjunto, estas capas trabajan de manera secuencial, donde la petición pasa desde el Controller al Service, luego al Repository, y finalmente a la base de datos, devolviendo la respuesta en sentido inverso. Esto permite una mejor organización, escalabilidad y mantenimiento del sistema.

Post en el microservicio Jean



Get

GET http://localhost:8080/api/v1/jean Send

Docs Params Authorization Headers 9 Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "precio": 29990,
3   "color": "Azul",
4   "talla": "M",
5   "tiro": "Medio",
6   "descripcion": "Jean clásico azul",
7   "marca": {
8     "id": 1
9   },
10  "modelo": {
11    "id": 1
12  }
13 }
```

Body Cookies Headers 9 Test Results 200 OK · 15 ms · 466 B

{ } JSON Preview Visualize

```
1 [
2   {
3     "id": 1,
4     "precio": 29990,
5     "color": "Azul",
6     "talla": "M",
7     "tiro": "Medio",
8     "descripcion": "Jean clásico azul",
9     "marca": {
10      "id": 1,
11      "nombre": "Levis"
12    },
13    "modelo": {
14      "id": 1,
15      "nombre": "Skinny"
16    }
17  }
18 ]
```

PUT

PUT http://localhost:8080/api/v1/jean/1 Send

Docs Params Authorization Headers 9 Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "precio": 34990,
3   "color": "Azul",
4   "talla": "M",
5   "tiro": "Alto",
6   "descripcion": "Jean actualizado precio",
7   "marca": {
8     "id": 1
9   },
10  "modelo": {
11    "id": 1
12  }
13 }
```

Body Cookies Headers 9 Test Results 200 OK · 10 ms · 469 B

{ } JSON Preview Visualize

```
1 {
2   "id": 1,
3   "precio": 34990,
4   "color": "Azul",
5   "talla": "M",
6   "tiro": "Medio",
7   "descripcion": "Jean actualizado precio",
8   "marca": {
9     "id": 1,
10    "nombre": "Levis"
11  },
12  "modelo": {
13    "id": 1,
14    "nombre": "Skinny"
15  }
16 }
```

GET POR ID

GET http://localhost:8080/api/v1/jean/1 Send

Docs Params Authorization Headers 9 Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "precio": 34990,
3   "color": "Azul",
4   "talla": "M",
5   "tiro": "Alto",
6   "descripcion": "Jean actualizado precio",
7   "marca": {
8     "id": 1
9   },
10  "modelo": {
11    "id": 1
12  }
13 }
```

Body Cookies Headers 9 Test Results 200 OK • 10 ms • 469 B

{ JSON Preview Visualize

```
1 {
2   "id": 1,
3   "precio": 34990,
4   "color": "Azul",
5   "talla": "M",
6   "tiro": "Medio",
7   "descripcion": "Jean actualizado precio",
8   "marca": {
9     "id": 1,
10    "nombre": "Levis"
11  },
12  "modelo": {
13    "id": 1,
14    "nombre": "Skinny"
15  }
16 }
```

DELETE

DELETE http://localhost:8080/api/v1/jean/1 Send

Docs Params Authorization Headers 9 Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "precio": 34990,
3   "color": "Azul",
4   "talla": "M",
5   "tiro": "Alto",
6   "descripcion": "Jean actualizado precio",
7   "marca": {
8     "id": 1
9   },
10  "modelo": {
11    "id": 1
12  }
13 }
```

Body Cookies Headers 9 Test Results 200 OK • 18 ms • 308 B

Raw Preview Visualize

```
1 Jean eliminado
```

Comunicación entre microservicios: ¿cómo se implementa?

La comunicación entre microservicios en el sistema Musa Jeans se realiza mediante peticiones HTTP REST utilizando herramientas como WebClient en Spring Boot. Esto permite que un microservicio pueda consumir la información de otro sin necesidad de compartir la misma base de datos. Por ejemplo, el servicio de pagos puede comunicarse con el servicio de clientes o ventas para obtener información necesaria al momento de procesar un pago. Este enfoque permite que cada microservicio sea independiente, escalable y fácil de mantener, ya que cada uno puede desarrollarse y desplegarse por separado.

Swagger

- **¿Qué es? ¿En qué consiste?**

Swagger es una herramienta que permite documentar y probar APIs REST de forma automática e interactiva. Su principal función es generar una interfaz gráfica donde se pueden visualizar todos los endpoints del sistema, junto con sus métodos HTTP, parámetros y respuestas. Además, permite ejecutar peticiones directamente desde el navegador sin necesidad de usar herramientas externas como Postman.

- **¿Cómo se implementa en microservicios Spring Boot?**

En un proyecto de microservicios con Spring Boot, Swagger se implementa mediante la librería **springdoc-openapi**. Esta dependencia permite generar automáticamente la documentación de cada microservicio.

Una vez agregada la dependencia en el pom.xml, Swagger se habilita automáticamente y se puede acceder a la interfaz web mediante una URL local, por ejemplo:

<http://localhost:8081/swagger-ui/index.html>

OpenAPI definition

v0OAS 3.0

/api/v1/jean/api-docs

Servers

http://localhost:8081 - Generated server uri

Jean

Operaciones relacionadas con la gestión de Jeans

^

GET	/api/v1/jean/{id}	Obtener un jean por su ID	▼
PUT	/api/v1/jean/{id}	Editar los datos del jean	▼
DELETE	/api/v1/jean/{id}	Borrar un jean	▼
GET	/api/v1/jean	Obtener todos los jeans	▼
POST	/api/v1/jean	Registra un jean en la base de datos	▼
GET	/api/v1/jean/talla/{talla}	Obtener todos los jeans por una talla	▼
GET	/api/v1/jean/marca/{nombre}	Obtener todos los jeans por marca	▼


```

service-jean > src > main > java > com > musa_jeans > service_jean > controller > JeanController.java > Language Support for Java(TM) by Red Hat > JeanController > listar()
1  package com.musa_jeans.service_jean.controller;
2
3  import java.util.List;
4
5  import org.springframework.beans.factory.annotation.Autowired;
6  import org.springframework.http.ResponseEntity;
7  import org.springframework.web.bind.annotation.CrossOrigin;
8  import org.springframework.web.bind.annotation.DeleteMapping;
9  import org.springframework.web.bind.annotation.GetMapping;
10 import org.springframework.web.bind.annotation.PathVariable;
11 import org.springframework.web.bind.annotation.PostMapping;
12 import org.springframework.web.bind.annotation.PutMapping;
13 import org.springframework.web.bind.annotation.RequestBody;
14 import org.springframework.web.bind.annotation.RequestMapping;
15 import org.springframework.web.bind.annotation.RestController;
16
17 import com.musa_jeans.service_jean.model.Jean;
18 import com.musa_jeans.service_jean.service.JeanService;
19
20 import io.swagger.v3.oas.annotations.Operation;
21 import io.swagger.v3.oas.annotations.tags.Tag;
22
23 @RestController
24 @CrossOrigin(origins = "**")
25 @RequestMapping("/api/v1/jean")
26 @Tag(name = "Jean", description = "Operaciones relacionadas con la gestión de Jeans")
27 public class JeanController {
28
29     @Autowired
30     private JeanService jeanService;
31
32     @Operation(summary = "Obtener todos los jeans", description = "Retorna una lista completa con todos los jeans registrados")
33     @GetMapping
34     public List<Jean> listar() {
35         return jeanService.listarTodos();
36     }
37
38     @Operation(summary = "Obtener un jean por su ID", description = "Retorna una lista con todos los datos de un jean")
39     @GetMapping("/{id}")
40     public ResponseEntity<Jean> obtener(@PathVariable Long id) {
41         return jeanService.buscarPorId(id)
42             .map(ResponseEntity::ok)
43             .orElse(ResponseEntity.notFound().build());
44     }
45
46     @Operation(summary = "Registra un jean en la base de datos", description = "Guarda un jean en la base de datos")
47     @PostMapping
48     public ResponseEntity<Jean> guardar(@RequestBody Jean jean) {
49         return ResponseEntity.ok(jeanService.guardar(jean));
50     }
51
52     @Operation(summary = "Editar los datos del jean", description = "Edita todos los datos del jean, buscando por ID")
53     @PutMapping("/{id}")
54     public ResponseEntity<Jean> actualizarJean(@PathVariable Long id, @RequestBody Jean jeanActualizado) {
55         Jean jean = jeanService.buscarPorId(id).orElse(other: null);
56
57         if (jean != null) {
58             jean.setPrecio(jeanActualizado.getPrecio());
59             jean.setDescripcion(jeanActualizado.getDescripcion());
60
61             jeanService.guardar(jean);
62             return ResponseEntity.ok(jean);
63         }
64     }
65 }

```

application<api-gateway> (tienda jeans) Java: Ready

OpenAPI definition

v0OAS 3.0

/api/v1/pagoapi-docs

Servers

http://localhost:8091 - Generated server url

Pago		Operaciones relacionadas con los pagos	^
GET	/api/v1/pago	Obtener todos los pagos	⌵
POST	/api/v1/pago	Registrar un pago	⌵
GET	/api/v1/pago/{id}	Obtener un pago por su ID	⌵
DELETE	/api/v1/pago/{id}	Eliminar un pago	⌵

```

1 package com.musa_jeans.service_pago.controller;
2
3 import java.util.List;
4
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.http.ResponseEntity;
7 import org.springframework.web.bind.annotation.CrossOrigin;
8 import org.springframework.web.bind.annotation.DeleteMapping;
9 import org.springframework.web.bind.annotation.GetMapping;
10 import org.springframework.web.bind.annotation.PathVariable;
11 import org.springframework.web.bind.annotation.PostMapping;
12 import org.springframework.web.bind.annotation.RequestBody;
13 import org.springframework.web.bind.annotation.RequestMapping;
14 import org.springframework.web.bind.annotation.RestController;
15
16 import com.musa_jeans.service_pago.model.Pago;
17 import com.musa_jeans.service_pago.service.PagoService;
18
19 import io.swagger.v3.oas.annotations.Operation;
20 import io.swagger.v3.oas.annotations.tags.Tag;
21
22 @RestController
23 @CrossOrigin(origins = "**")
24 @RequestMapping("api/v1/pago")
25 @Tag(name = "Pago", description = "Operaciones relacionadas con los pagos")
26 public class PagoController {
27
28     @Autowired
29     private PagoService pagoService;
30
31     @Operation(summary = "Obtener todos los pagos", description = "Retorna una lista completa de todos los pagos registrados")
32     @GetMapping
33     public List<Pago> listar() {
34         return pagoService.listarTodos();
35     }
36
37     @Operation(summary = "Obtener un pago por su ID", description = "Retorna una lista con un pago en especifico")
38     @GetMapping("/{id}")
39     public ResponseEntity<Pago> obtener(@PathVariable Long id) {
40         Pago pago = pagoService.buscarPorId(id).orElse(other: null);
41
42         if (pago == null) {
43             return ResponseEntity.notFound().build();
44         }
45         return ResponseEntity.ok(pago);
46     }
47
48     @Operation(summary = "Registrar un pago", description = "Ingresa un pago a la base de datos")
49     @PostMapping
50     public ResponseEntity<Pago> guardar(@RequestBody Pago pago) {
51         return ResponseEntity.ok(pagoService.guardar(pago));
52     }
53
54     @Operation(summary = "Eliminar un pago", description = "Elimina un pago por su ID")
55     @DeleteMapping("/{id}")
56     public ResponseEntity<Void> eliminar(@PathVariable Long id) {
57         Pago pago = pagoService.buscarPorId(id).orElse(other: null);
58     }
59 }

```

OpenAPI definition

v0OAS 3.0

/api/v1/envio/api-docs

Servers

http://localhost:8092 - Generated server url

EnvíoOperaciones relacionadas con los envíos

GET	/api/v1/envio/{id}	Obtener un envío por su ID	▼
PUT	/api/v1/envio/{id}	Actualizar un envío	▼
DELETE	/api/v1/envio/{id}	Eliminar un envío	▼
GET	/api/v1/envio	Obtener todos los envíos	▼
POST	/api/v1/envio	Registrar un envío	▼

```

service_envio > src > main > java > com > musa_jeans > service_envio > controller > EnvioController.java > Language Support for Java(TM) by Red Hat > EnvioController > eliminar(Long)
1  package com.musa_jeans.service_envio.controller;
2
3  import java.util.List;
4
5  import org.springframework.beans.factory.annotation.Autowired;
6  import org.springframework.http.ResponseEntity;
7  import org.springframework.web.bind.annotation.CrossOrigin;
8  import org.springframework.web.bind.annotation.DeleteMapping;
9  import org.springframework.web.bind.annotation.GetMapping;
10 import org.springframework.web.bind.annotation.PathVariable;
11 import org.springframework.web.bind.annotation.PostMapping;
12 import org.springframework.web.bind.annotation.PutMapping;
13 import org.springframework.web.bind.annotation.RequestBody;
14 import org.springframework.web.bind.annotation.RequestMapping;
15 import org.springframework.web.bind.annotation.RestController;
16
17 import com.musa_jeans.service_envio.model.Envio;
18 import com.musa_jeans.service_envio.service.EnvioService;
19
20 import io.swagger.v3.oas.annotations.Operation;
21 import io.swagger.v3.oas.annotations.tags.Tag;
22
23 @RestController
24 @CrossOrigin(origins = "**")
25 @RequestMapping("/api/v1/envio")
26 @Tag(name = "Envio", description = "Operaciones relacionadas con los envíos")
27 public class EnvioController {
28
29     @Autowired
30     private EnvioService envioService;
31
32     @Operation(summary = "Obtener todos los envíos", description = "Retorna una lista completa de todos los envíos registrados")
33     @GetMapping
34     public List<Envio> listar() {
35         return envioService.listarTodos();
36     }
37
38     @Operation(summary = "Obtener un envío por su ID", description = "Retorna un envío específico")
39     @GetMapping("/{id}")
40     public ResponseEntity<Envio> obtener(@PathVariable Long id) {
41
42         Envio envio = envioService.buscarPorId(id).orElse(other: null);
43
44         if (envio == null) {
45             return ResponseEntity.notFound().build();
46         }
47
48         return ResponseEntity.ok(envio);
49     }
50
51     @Operation(summary = "Registrar un envío", description = "Ingresa un envío a la base de datos")
52     @PostMapping
53     public ResponseEntity<Envio> guardar(@RequestBody Envio envio) {
54         return ResponseEntity.ok(envioService.guardar(envio));
55     }
56
57     @Operation(summary = "Actualizar un envío", description = "Actualiza un envío por su ID")
58     @PutMapping("/{id}")

```

Testing

- **¿En qué consiste? ¿Por qué es importante testear?**

El testing es el proceso de verificar que el software funciona correctamente antes de ser desplegado. Su importancia radica en que permite detectar errores en etapas tempranas del desarrollo, evitando fallos en producción. Además, mejora la calidad del sistema, asegura el correcto funcionamiento de la lógica de negocio y facilita el mantenimiento del código.

- **Pruebas unitarias (AAA)**

Las pruebas unitarias se basan en el modelo AAA:

- **Arrange (Preparar):** se configuran los datos necesarios para la prueba.
- **Act (Ejecutar):** se ejecuta el método que se quiere probar.
- **Assert (Verificar):** se comprueba que el resultado sea el esperado.

Este enfoque permite estructurar los tests de manera clara y ordenada.

- **Evidencias**

Test guardar un jean

```
35  @Test
36  @DisplayName("Debería guardar un jean correctamente")
37  void guardarJeanTest() {
38
39      Marca marca = new Marca();
40      marca.setId(id: 1L);
41      marca.setNombre(nombre: "Levis");
42
43      Modelo modelo = new Modelo();
44      modelo.setId(id: 1L);
45      modelo.setNombre(nombre: "Skinny");
46
47      Jean jean = new Jean();
48      jean.setPrecio(precio: 29990);
49      jean.setColor(color: "Azul");
50      jean.setTalla(talla: "XL");
51      jean.setTiro(tiro: "Medio");
52      jean.setDescripcion(descripcion: "Jean clásico");
53      jean.setMarca(marca);
54      jean.setModelo(modelo);
55
56      when(jeanRepository.save(any(Jean.class))).thenReturn(invocation -> {
57          Jean j = invocation.getArgument(0);
58          j.setId(id: 1L);
59          return j;
60      });
61
62      Jean resultado = jeanService.guardar(jean);
63
64      assertNotNull(resultado);
65      assertEquals(expected: 1L, resultado.getId());
66
67      assertEquals(expected: 29990, resultado.getPrecio());
68      assertEquals(expected: "Azul", resultado.getColor());
69      assertEquals(expected: "XL", resultado.getTalla());
70      assertEquals(expected: "Medio", resultado.getTiro());
71      assertEquals(expected: "Jean clásico", resultado.getDescripcion());
72
73      assertEquals(expected: "Levis", resultado.getMarca().getNombre());
74      assertEquals(expected: "Skinny", resultado.getModelo().getNombre());
75
76      verify(jeanRepository, times(1)).save(jean);
77  }
78
```

The screenshot shows an IDE window with a 'TEST RESULTS' tab. The left pane lists test results for 'JeanServiceTest' with the following entries:

- %TESTS 3, buscarPorMarcaNombreTest(com.musa_jeans.service_jean.service.JeanServiceTest)
- %TESTE 3, buscarPorMarcaNombreTest(com.musa_jeans.service_jean.service.JeanServiceTest)
- %TESTS 4, eliminarTest(com.musa_jeans.service_jean.service.JeanServiceTest)
- %TESTE 4, eliminarTest(com.musa_jeans.service_jean.service.JeanServiceTest)
- %TESTS 5, buscarPorIdTest(com.musa_jeans.service_jean.service.JeanServiceTest)
- %TESTE 5, buscarPorIdTest(com.musa_jeans.service_jean.service.JeanServiceTest)
- %TESTS 6, guardarJeanTest(com.musa_jeans.service_jean.service.JeanServiceTest)
- %TESTE 6, guardarJeanTest(com.musa_jeans.service_jean.service.JeanServiceTest)

The right pane shows the 'Test Runner for Java' window. It lists the tests and their status:

- ✓ JeanServiceTest \$(symbol-namespace) com.musa_jeans.service_jean.service + \$(project) service-jean
- ✓ guardarJeanTest0 \$(symbol-class) JeanServiceTest + \$(symbol-namespace) com.musa_jeans.service_jean.service + \$(project) service-jean
- ✓ buscarPorIdTest0 \$(symbol-class) JeanServiceTest + \$(symbol-namespace) com.musa_jeans.service_jean.service + \$(project) service-jean
- ✓ eliminarTest0 \$(symbol-class) JeanServiceTest + \$(symbol-namespace) com.musa_jeans.service_jean.service + \$(project) service-jean
- ✓ buscarPorMarcaNombreTest0 \$(symbol-class) JeanServiceTest + \$(symbol-namespace) com.musa_jeans.service_jean.service + \$(project) service-jean

Below the list, it shows '1 older result' and 'Test run at 6/21/2026, 10:05:52 PM'.

At the bottom, the status bar shows 'Ln 48, Col 22 Tab Size: 4 UTF-8 CRLF {} XML Sign In Prettier'.

Registrar y eliminar cliente

```

91     @Test
92     @DisplayName("Debería registrar un cliente correctamente")
93     void registrarClienteTest() {
94
95         Cliente cliente = new Cliente();
96         cliente.setRut(rut: "20051806-3");
97         cliente.setNombre(nombre: "Felipe");
98         cliente.setCorreo(correo: "felipe@gmail.com");
99         cliente.setDireccion(direccion: "Maipú");
100
101         Cliente clienteGuardado = new Cliente();
102         clienteGuardado.setId(id: 1L);
103         clienteGuardado.setRut(rut: "20051806-3");
104         clienteGuardado.setNombre(nombre: "Felipe");
105         clienteGuardado.setCorreo(correo: "felipe@gmail.com");
106         clienteGuardado.setDireccion(direccion: "Maipú");
107
108         when(clienteRepository.save(any(Cliente.class))).thenReturn(clienteGuardado);
109
110         Cliente resultado = clienteService.registrarCliente(cliente);
111
112         assertNotNull(resultado);
113         assertEquals(expected: 1L, resultado.getId());
114         assertEquals(expected: "Felipe", resultado.getNombre());
115         assertEquals(expected: "20051806-3", resultado.getRut());
116
117         verify(clienteRepository, times(1)).save(cliente);
118     }
119
120     @Test
121     @DisplayName("Debería eliminar un cliente por ID")
122     void eliminarClienteTest() {
123
124         Long id = 1L;
125         doNothing().when(clienteRepository).deleteById(id);
126         String resultado = clienteService.eliminarCliente(id);
127         assertEquals(expected: "Cliente eliminado", resultado);
128         verify(clienteRepository, times(1)).deleteById(id);
129     }
130 }

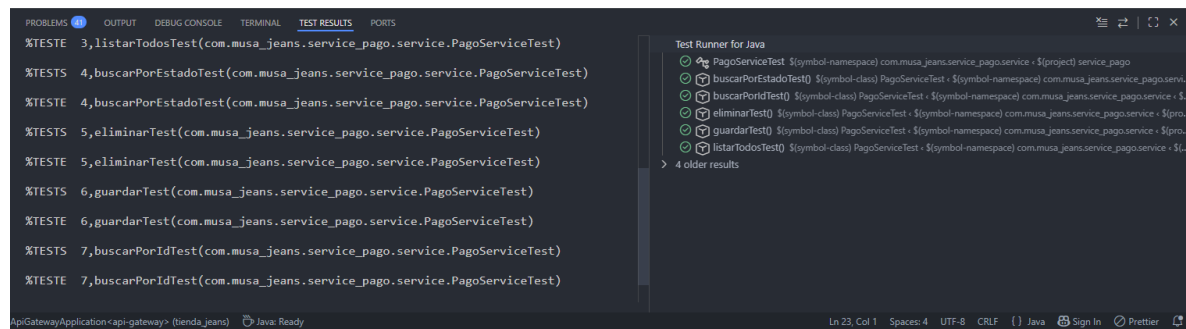
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS
false,1,false,2,Debería eliminar un cliente por ID, [engine:junit-jupiter][@class:com.musa_jeans.service_cliente.service.ClienteServiceTest][@method:eliminarClienteTest()]
%TESTS 3, listarTodosTest(com.musa_jeans.service_cliente.service.ClienteServiceTest)
%TESTE 3, listarTodosTest(com.musa_jeans.service_cliente.service.ClienteServiceTest)
%TESTS 4, buscarPorRutTest(com.musa_jeans.service_cliente.service.ClienteServiceTest)
%TESTE 4, buscarPorRutTest(com.musa_jeans.service_cliente.service.ClienteServiceTest)
%TESTS 5, registrarClienteTest(com.musa_jeans.service_cliente.service.ClienteServiceTest)
%TESTE 5, registrarClienteTest(com.musa_jeans.service_cliente.service.ClienteServiceTest)
%TESTS 6, eliminarClienteTest(com.musa_jeans.service_cliente.service.ClienteServiceTest)
%TESTE 6, eliminarClienteTest(com.musa_jeans.service_cliente.service.ClienteServiceTest)
%RUNTIME1113

Test Runner for Java
  ClienteServiceTest $[symbol-namespace] com.musa_jeans.service_cliente.service : $[project] service_cliente
    buscarPorRutTest0 $[symbol-class] ClienteServiceTest : $[symbol-namespace] com.musa_jeans.service_cliente.serv
    eliminarClienteTest0 $[symbol-class] ClienteServiceTest : $[symbol-namespace] com.musa_jeans.service_cliente.serv
    listarTodosTest0 $[symbol-class] ClienteServiceTest : $[symbol-namespace] com.musa_jeans.service_cliente.servi
    registrarClienteTest0 $[symbol-class] ClienteServiceTest : $[symbol-namespace] com.musa_jeans.service_cliente.serv
    > 2 older results
```


Buscar pagos por su estado

```
126
127 @Test
128 @DisplayName("Debería buscar pagos por estado")
129 void buscarPorEstadoTest() {
130
131     String estado = "COMPLETADO";
132
133     List<Pago> lista = new ArrayList<>();
134
135     Pago pago = new Pago();
136     pago.setId(id: 1L);
137     pago.setEstado(estado);
138
139     lista.add(pago);
140
141     when(pagoRepository.findByEstadoIgnoreCase(estado)).thenReturn(lista);
142
143     List<Pago> resultado = pagoService.buscarPorEstado(estado);
144
145     assertNotNull(resultado);
146     assertEquals(expected: 1, resultado.size());
147     assertEquals(estado, resultado.get(index: 0).getEstado());
148
149     verify(pagoRepository, times(1)).findByEstadoIgnoreCase(estado);
150 }
151 }
```



The screenshot shows an IDE window with the 'TEST RESULTS' tab selected. The left pane displays a list of test results, each preceded by a status icon (green for passed, red for failed). The tests listed are:

- 3, listarTodosTest(com.musa_jeans.service_pago.service.PagoServiceTest)
- 4, buscarPorEstadoTest(com.musa_jeans.service_pago.service.PagoServiceTest)
- 4, buscarPorEstadoTest(com.musa_jeans.service_pago.service.PagoServiceTest)
- 5, eliminarTest(com.musa_jeans.service_pago.service.PagoServiceTest)
- 5, eliminarTest(com.musa_jeans.service_pago.service.PagoServiceTest)
- 6, guardarTest(com.musa_jeans.service_pago.service.PagoServiceTest)
- 6, guardarTest(com.musa_jeans.service_pago.service.PagoServiceTest)
- 7, buscarPorIdTest(com.musa_jeans.service_pago.service.PagoServiceTest)
- 7, buscarPorIdTest(com.musa_jeans.service_pago.service.PagoServiceTest)

The right pane shows the 'Test Runner for Java' window, which displays a list of test classes and methods, each with a status icon. The tests listed are:

- PagoServiceTest
- buscarPorEstadoTest()
- buscarPorIdTest()
- eliminarTest()
- guardarTest()
- listarTodosTest()

At the bottom of the IDE window, the status bar shows 'Ln 23, Col 1', 'Spaces: 4', 'UTF-8', 'CRLF', 'Java', and 'Sign In'.